

Balance Sheet Exposure Engine

Technical Documentation and Code Reference

Document type Technical / operational PDF documentation	Source analyzed BS EXPOSURE ENGINE - EXECUTABLE SCRIPT.py
Prepared for Treasury Reporting / Balance Sheet Exposure workflow	Prepared on 23 April 2026

This document explains how the Balance Sheet Exposure engine works, what problems it solves, which inputs it requires, how the runtime decisions affect the output, and how the script should be maintained or operated safely.

Design standard
Primary headings use the requested dark treasury color (#2D257D). Body typography uses an Arial-compatible sans-serif family for clean operational readability.

Table of Contents

Executive Summary

3

Business Problem and Value Delivered

3

What the Engine Does - Intuition for Non-Coders

3

Technical Scope and Runtime Environment

3

Required Inputs

4

Major Components and Architecture

4

End-to-End Processing Logic

5

Data Transformation Rules

5

Workbook Objects Updated by the Engine

6

Classification Logic: IS_IN_ANALYSIS

6

Outputs Produced

6

Error Handling, Logging, and Recoverability

7

Assumptions and Limitations

7

Maintenance Guidance

7

Function Reference

8

Logging, validation and user input

8

Excel and dataframe ingestion

8

Date and schema preparation

8

Excel COM and table management

9

Formatting and enrichment

9

Interactive classification support

10

Statistics sheet helpers

10

Execution wrappers

10

Document objective

Provide a professional, end-to-end explanation of what the script does, why it exists, how it processes data, what workbook objects it updates, and how each function contributes to the overall engine.

Executive Summary

The Balance Sheet Exposure Generator is a Windows-based Python automation engine designed to update revaluation masterdata, enrich it with Chart of Accounts attributes, classify exposure-relevant accounts, and populate a reporting workbook with an auditable current-period Balance Sheet Exposure dataset.

At a business level, the engine reduces a repetitive treasury reporting process into a guided console workflow. Instead of manually merging monthly files, forcing schema consistency, updating workbook tables, enriching account attributes, and checking audit statistics, the user supplies source files and classification decisions while the engine performs the heavy processing in a controlled sequence.

Business Problem and Value Delivered

The script addresses five recurring reporting pain points. First, historical and current revaluation files often arrive in different months and need to be merged into a single masterdata structure. Second, file headers can vary between exports, so a strict column match is not reliable. Third, the reporting workbook expects a structured Excel table and specific sheets to exist. Fourth, account-level analysis scope has to be controlled through a repeatable CONSIDERED versus NOT_CONSIDERED logic. Fifth, the team needs an audit trail showing what was loaded, how it was classified, and what date range the final masterdata now covers.

The engine solves these issues by standardizing headers, aligning every new file to the base schema, forcing reporting dates into text-safe DD/MM/YYYY values, enriching rows from the Chart of Accounts, creating or updating Excel tables directly through Excel COM, and generating a statistics sheet that summarizes the full data-processing outcome.

What the Engine Does - Intuition for Non-Coders

Imagine the script as a controlled assembly line.

The first station checks that the user provided the correct source files. The second station reads the historical revaluation masterdata and remembers its column structure. The third station takes any historical catch-up files and current-period files, stamps them with the right entity and month-end date, and reshapes them to match the masterdata structure. The fourth station enriches accounts with group and type information from the Chart of Accounts and asks the user how certain accounts should be treated in the report. The fifth station opens the reporting workbook, replaces the revaluation master table, updates audit sheets, and exports both the final working report and a fresh masterdata file that can be reused next month.

Technical Scope and Runtime Environment

The script is explicitly written for a Windows environment with a locally installed desktop version of Microsoft Excel. It uses xlwings and Win32 COM automation to open, modify, and save the reporting workbook. It also expects pandas plus either XlsxWriter or openpyxl for standalone file exports. When packaged as an executable, it preserves multiprocessing freeze support and writes runtime logs under the user's LOCALAPPDATA folder.

This means the engine is not platform-neutral. It is intended for treasury users working on Windows machines with Excel available and permission to open, edit, and save the relevant files.

Required Inputs

The engine requires three mandatory starting files: a current revaluation masterdata file, a reporting workbook, and a Chart of Accounts workbook. It may also require optional outdated source files for historical backfilling and current-month update files for the period being reported. In addition, the user must supply the report year and month, and in some cases company labels and account classification decisions.

From a process perspective, the most important distinction is this: outdated files are historical backfill files and receive their own explicit historical month-end date, while current files are stamped with the report month-end chosen by the user and are used to build the current reporting cycle.

Input category	Examples	Why it matters
Mandatory base files	Current revaluation masterdata; reporting workbook; Chart of Accounts	These three files are always needed to build the updated report.
Optional historical files	Older monthly revaluation files that were not yet appended	These files backfill historical month-end data into the masterdata.
Current update files	The source files belonging to the report month	These files update the masterdata for the current reporting cycle.
Runtime decisions	Report year/month; company labels; account scope overrides	These decisions affect the assigned dates, entity tags, and IS_IN_ANALYSIS values.

Major Components and Architecture

The script can be read in six major layers.

Layer 1 is startup protection: logging setup, exception capture, and wrapper logic. Layer 2 is user interaction: prompts, validation, and company detection. Layer 3 is dataframe preparation: header normalization, date normalization, schema alignment, and total-row cleanup. Layer 4 is Excel workbook control: sheet creation, table resizing, number formatting, uppercase formatting, and gate-status updates. Layer 5 is business enrichment: COA mapping and IS_IN_ANALYSIS assignment. Layer 6 is output generation: statistics sheet creation, report save, and export of a reusable masterdata workbook.

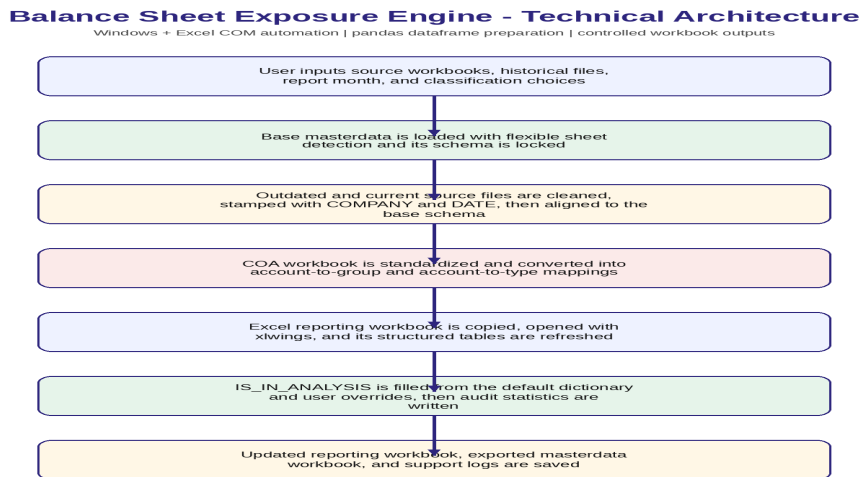


Figure 1. High-level architecture of the Balance Sheet Exposure engine.

End-to-End Processing Logic

The workflow starts by reading the base masterdata with flexible sheet detection. The engine tries MASTERDATA_REVAL first, then known fallback sheet names, then the first worksheet if necessary. The base dataset is normalized, and its column order becomes the target schema for all new files.

Any outdated files are then loaded one by one. Each file is cleaned, stamped with COMPANY and its own month-end DATE, and aligned to the base schema. Current files follow the same path, except that they all receive the report month-end selected by the user. The engine concatenates base data plus outdated data plus current data into one final dataframe.

After that, the script creates a working copy of the reporting workbook, opens Excel through xlwings, writes a standardized Chart of Accounts sheet, updates the REVAL_MASTER_DATA_1 table, enriches account group and type values, asks the user about analysis-scope overrides, writes the DATA_PROCESS_STATS audit sheet, saves the workbook, and finally exports a fresh standalone masterdata file for future reuse.

Data Transformation Rules

Several transformation rules are especially important.

Header matching is tolerant. The script standardizes incoming column names by uppercasing them, replacing punctuation with underscores, and removing ambiguity through `uniquify()`. This is why slightly different headers such as Account, ACCOUNT NO, or Account No. can still map to a canonical ACCOUNT field.

Dates are forced into DD/MM/YYYY text. This is done deliberately so Excel does not reinterpret dates in a locale-sensitive way. When the engine writes the DATE column into Excel, it prefixes values with an apostrophe to preserve the text representation.

Account matching is also normalized. The COA maps and IS_IN_ANALYSIS logic rely on digit-only account keys, which allows the script to reconcile account values even when they appear with formatting differences.

Workbook Objects Updated by the Engine

Inside the reporting workbook, the script interacts with specific objects. It updates the gate cell BS_EXPOSURE_REPORT!H5 to reflect IN PROGRESS and DONE status. It ensures a CHART_OF_ACCOUNTS worksheet exists and writes a COA_DATA table. It ensures the REVAL_MASTER_DATA_1 worksheet contains the MASTERDATA1 table and resizes that table to match the current final dataframe. It also recreates the DATA_PROCESS_STATS worksheet at the end of the process.

In practice, this means the workbook acts as both a reporting shell and a structured data container. The script does not only paste raw values; it deliberately maintains table-based structures so downstream formulas, visuals, and workbook logic have a predictable data source.

Workbook object	Role in the process
BS_EXPOSURE_REPORT!H5	Gate cell switched to IN PROGRESS during the run and DONE after successful completion.
CHART_OF_ACCOUNTS / COA_DATA	A refreshed worksheet and Excel table containing the standardized Chart of Accounts.
REVAL_MASTER_DATA_1 / MASTERDATA1	Primary destination table holding the updated and enriched revaluation masterdata.
DATA_PROCESS_STATS	Audit worksheet showing row counts, company counts, date ranges, classification counts, and column data types.

Classification Logic: IS_IN_ANALYSIS

The IS_IN_ANALYSIS process is one of the most important business rules in the engine.

The script starts with a built-in dictionary mapping account numbers to CONSIDERED or NOT_CONSIDERED. The user can keep this setup as-is or override selected accounts during runtime. If the data contains accounts not present in the built-in dictionary, the script can also ask the user to classify these newly discovered accounts one by one. The resulting values are written into the IS_IN_ANALYSIS column of the masterdata table.

This makes the engine both governed and flexible. Governance comes from the default dictionary; flexibility comes from the runtime override process.

Outputs Produced

The engine produces four key outputs. First, it saves a Python-generated working copy of the reporting workbook in the same folder as the supplied reporting file. Second, it exports a standalone MASTERDATA_YYYYMMM_PYTHON_GENERATED.xlsx file that uses the MASTERDATA_REVAL sheet name for future compatibility. Third, it writes a runtime log file under %LOCALAPPDATA%\ExposureAutomationLogs\exposure_automation_run.log. Fourth, if an unhandled error occurs, it writes exposure_last_error.txt in the run directory and attempts to show a Windows message box.

Operationally, the most important deliverables are the updated report workbook and the exported masterdata workbook. The log and error files are support artifacts for debugging and remote user assistance.

Error Handling, Logging, and Recoverability

The engine is designed to fail loudly and informatively. The `abort()` helper is used whenever the script detects invalid inputs or environmental issues. The `write_error_and_notify()` helper captures unexpected exceptions, writes full tracebacks to disk, and shows a Windows message box so the user is not left with a silent failure. The `main_wrapper()` function preserves this behavior even in executable form.

This design matters because the process depends on Excel COM, user file paths, and local runtime libraries. When something breaks, the log file and last-error text file give the reporting team a clear starting point for troubleshooting.

Assumptions and Limitations

The script assumes a Windows environment, access to Microsoft Excel, a valid revaluation masterdata file, and a COA that contains account-level mapping fields. It also assumes that the first sheet of each incoming source file is the relevant data sheet. The `drop_trailing_total_row()` logic assumes the final non-empty row of source extracts is a Total row and removes it. If a source file does not follow that pattern, the user should validate the input before running the engine.

A second limitation is that company detection relies on filename patterns unless the user types the company manually. Consistent source-file naming therefore improves reliability and reduces manual intervention.

Maintenance Guidance

When updating the script in the future, focus on four areas first. If source-file headers change, adjust the header normalization and column-detection logic. If new entities are introduced, extend the company-detection patterns. If the classification policy changes, update the built-in `DICT_IS_IN_ANALYSIS` and communicate the new rules to users. If workbook structures change, confirm that the destination sheet names, table names, and gate cell still match the report template.

For monthly operations, the safest practice is to keep the exported Python-generated masterdata as the next cycle's base masterdata. The script was explicitly revised to support that feedback loop.

Function Reference

The following appendix summarizes every function defined in the script. The purpose is operational orientation, not line-by-line code review. Grouping the functions by responsibility makes maintenance easier and clarifies where future modifications should be made.

Logging, validation and user input

Function	Purpose
write_error_and_notify	Writes unhandled exceptions to disk, logs them, and shows a Windows message box.
slog	Prints and logs step-level progress messages.
abort	Stops execution with a controlled error message and non-zero exit.
safe_input	Reads console input and aborts cleanly on EOF or keyboard interruption.
ensure_ext	Validates that a supplied path has an allowed file extension.
prompt_path	Prompts for a file path, validates existence and extension, and returns an absolute path.
prompt_yesno	Prompts until a YES/NO answer is supplied.
prompt_int	Prompts until a non-negative integer is supplied.
prompt_year	Validates a four-digit reporting year.
prompt_month	Accepts month numbers or month names and converts them to a numeric month.
detect_company_from_filename	Detects the entity from the filename or asks the user to provide it manually.
unique_copy_path_same_folder	Builds a non-conflicting output path for the Python-generated reporting workbook.

Excel and dataframe ingestion

Function	Purpose
read_excel_first_sheet	Reads the first worksheet of an Excel file into a dataframe.
read_excel_named_sheet	Reads a specifically named worksheet into a dataframe.
read_excel_sheet_auto	Reads the preferred master sheet, then tries fallback sheet names before failing.
normalize_headers	Trims and normalizes dataframe column headers.
drop_trailing_total_row	Removes the final non-empty row, assuming it is a trailing Total row from IFS exports.

Date and schema preparation

Function	Purpose
excel_serial_to_datetime	Converts Excel serial dates to pandas timestamps.
_fmt_ddmmyyyy	Formats a timestamp as DD/MM/YYYY.

Function	Purpose
_format_to_ddmmyyyy_series	Normalizes a whole date column into DD/MM/YYYY text.
last_day_of_month	Returns the last calendar day of a given month.
build_lastday_ddmmyyyy	Builds the month-end date string used for appended source files.
_detect_date_col_name	Detects the most likely date column in a dataframe.
prepare_base_master_ddmmyyyy	Moves COMPANY and DATE to the front and ensures DATE is text-ready.
mark_date_as_text	Prefixes DATE values with an apostrophe so Excel preserves them as text.
align_to_schema	Aligns incoming files to the base masterdata schema using normalized headers.
std_header	Standardizes a header name into a machine-safe uppercase token.
uniquify	Ensures duplicate standardized headers become unique.

Excel COM and table management

Function	Purpose
import_xlwings	Loads xlwings and aborts with guidance if it is unavailable.
set_perf_mode	Turns Excel performance options on or off during bulk writes.
set_gate_value	Writes a status value into the report gate cell.
get_or_create_sheet	Returns an existing worksheet or creates it at the workbook end.
find_listobject_by_name	Finds an Excel table (ListObject) by name.
create_table_on_used_range	Creates a new Excel table over the current used range.
ensure_table_masterdata1	Ensures the REVAL masterdata destination sheet and table exist.
read_table_headers	Reads the header row coordinates and header labels from an Excel table.
table_size	Returns the current data row and column count of an Excel table.
_find_col_index	Finds a specific header position inside a header list.
_enforce_date_text_format	Applies text format to the DATE column in Excel so dates do not auto-convert.
_to_writeable_values	Converts a dataframe to write-ready row values and safeguards DATE as text.
resize_table_and_write	Resizes an Excel table to the new shape and writes headers and values.
_resolve_xlsx_engine	Detects whether XlsxWriter or openpyxl is available for export.
excel_writer	Creates an ExcelWriter using the resolved engine.
export_masterdata_table	Exports the updated masterdata table to a standalone Excel file.

Formatting and enrichment

Function	Purpose
apply_number_format_for_headers	Applies predefined numeric formats to selected columns.

Function	Purpose
uppercase_text_columns	Uppercases selected text columns directly in Excel.
standardize_headers_df	Standardizes dataframe headers using the shared header normalizer.
standardize_coa_dataframe	Normalizes the Chart of Accounts sheet and uppercases descriptive columns.
to_digit_key	Converts account-like values into digit-only matching keys.
find_best_column	Detects the best matching column based on preferred names and token matching.
build_coa_maps	Builds account-to-group and account-to-type lookup dictionaries from the COA.
normalize_account_key	Alias helper for account normalization.
standardize_table_headers_in_place	Standardizes the Excel table headers in place.
ensure_columns_exist_at_end	Adds missing output columns to the Excel table.
read_table_as_dataframe	Reads an Excel table back into a dataframe for downstream logic.
find_account_header_idx	Finds the primary ACCOUNT column among standardized headers.
fill_and_write_analysis_cols	Populates ACCOUNT_GROUP_DESCRIPTION, ACCOUNT_TYPE, and IS_IN_ANALYSIS in Excel.

Interactive classification support

Function	Purpose
prompt_show_current_setup	Prints the current account classification setup to the console.
interactive_override_is_in	Collects user overrides for IS_IN_ANALYSIS and optionally labels newly found accounts.

Statistics sheet helpers

Function	Purpose
_strip_apostrophe	Removes the leading apostrophe used to preserve text-formatted dates.
_parse_date_series_ddmmyyyy	Parses DD/MM/YYYY date text into pandas timestamps.
_classify_dtype	Classifies a column as text, number, date, or empty.
_add_df_as_table	Writes a dataframe block into Excel and turns it into a table.
build_and_write_stats_sheet	Creates the DATA_PROCESS_STATS worksheet and populates audit summaries.

Execution wrappers

Function	Purpose
main	Runs the full Balance Sheet Exposure workflow from prompts to final outputs.
main_wrapper	Adds freeze_support, global exception handling, and safe exit behavior.